Bits	Description	Type	Reset
4	**EP2_TRANSACTION**	WC	0x0
3	**EP1_SEQ**	WC	0x0
2	**EP1_TRANSACTION**	WC	0x0
1	**EP0_SEQ**	WC	0x0
0	**EP0_TRANSACTION**	WC	0x0

USB: DEV_SM_WATCHDOG Register

Offset: 0x114

Description

Watchdog that forces the device state machine to idle and raises an interrupt if the device stays in a state that isn't idle for the configured limit. The counter is reset on every state transition.
Set limit while enable is low and then set the enable.

Table 1226. DEV_SM_WATCHDOG Register

Bits	Description	Type	Reset
31:21	Reserved.	-	-
20	**FIRED**	WC	0x0
19	**RESET**: Set to 1 to forcibly reset the device state machine on watchdog expiry	RW	0x0
18	**ENABLE**	RW	0x0
17:0	**LIMIT**	RW	0x00000

12.8. System timers

12.8.1. Overview

The system timer peripheral on RP2350 provides a microsecond timebase for the system, and generates interrupts based on this timebase. RP2350 has two instances of the system timer: `TIMER0` and `TIMER1`. This allows for two separately controlled timers, each in a different security domain. It supports the following features:

- A single 64-bit counter, incrementing once per microsecond
  - Read from a pair of latching registers for race-free reads over a 32-bit bus
- Four alarms that match on the lower 32 bits of the counter and generate IRQ on match

The timer uses a one microsecond reference generated by the tick generators (see Section 8.5), and derived from the reference clock (Figure 33), which itself is usually connected directly to the crystal oscillator (Section 8.2).

The 64-bit counter effectively cannot overflow (thousands of years at 1 MHz), so the system timer is completely monotonic in practice.

12.8.1.1. Changes from RP2040

- RP2350 now has two timer instances: `TIMER0` and `TIMER1`
- On RP2350, the tick source for each timer comes from the system-level tick generators (see Section 8.5)
- RP2350 added two new registers: `LOCKED` is used to disable write access to the timer, and `SOURCE` allows the timer to

count system clock cycles rather than a 1 µs tick

12.8.1.2. Other timer resources on RP2350

The system timer provides a global timebase for software. RP2350 has a number of other programmable counter resources which can provide regular interrupts, or trigger DMA transfers.

- The PWM (Section 12.5) contains 12× 16-bit programmable counters. These counters:
  - run at up to system speed
  - can generate interrupts to either of two system IRQ lines
  - can be continuously reprogrammed via the DMA
  - can trigger DMA transfers to other peripherals
- 12× PIO state machines (Chapter 11) can count 32-bit values at system speed, and generate interrupts.
- The DMA (Section 12.6) has four internal pacing timers which trigger transfers at regular intervals.
- Each Cortex-M33 core (Section 3.7) has a standard 24-bit SysTick timer, counting either the microsecond tick (Section 8.5) or the system clock.
- SIO has a standard 64-bit RISC-V platform timer (Section 3.1.8). Arm and RISC-V software can use this timer.
- The Power Manager (Chapter 6) incorporates a 64-bit timer (AON Timer) which nominally counts milliseconds (see Section 12.10). This is the only timer that runs when the chip is in its lowest power state, with all switchable power domains powered down. It is used to schedule power-ups.

12.8.2. Counter

The timer has a 64-bit counter, but RP2350 only has a 32-bit data bus. This means that the `TIME` value is accessed through a pair of registers. These are:

- `TIMEHW` and `TIMELW` to write the time
- `TIMEHR` and `TIMELR` to read the time

To use these pairs, access the lower register, `L`, followed by the higher register, `H`. In the read case, reading the `L` register latches the value in the `H` register to provide an accurate time. To read the raw time without any latching, use `TIMERAWH` and `TIMERAWL`.

CAUTION

Don't write to the `TIMEHW` and `TIMELW` registers to force a new time value if other software may be using the timer. The SDK uses the time value for timeouts, elapsed time, and more, and expects the value to increase monotonically.

12.8.3. Alarms

The timer has 4 alarms, and outputs a separate interrupt for each alarm. The alarms match on the lower 32 bits of the 64-bit counter, which means they can be fired at a maximum of 2^{32} microseconds into the future. This is equivalent to:

- $2^{32} \div 10^6$: ~4295 seconds
- 4295 ÷ 60: ~72 minutes

NOTE

This timer supports alarm intervals on the order of one microsecond to one hour. For a longer alarm, see Section 12.10.

To enable an alarm:

1. Enable the interrupt at the timer with a write to the appropriate alarm bit in INTE (e.g. `(1 << 0)` for `ALARM0`).
2. Enable the appropriate timer interrupt at the processor (see Section 3.2).
3. Write the time you would like the interrupt to fire to `ALARM0` (i.e. the current value in `TIMERAWL` plus your desired alarm time in microseconds). Writing the time to the `ALARM` register sets the `ARMED` bit as a side effect.

Once the alarm has fired, the `ARMED` bit clears to `0`. To clear the latched interrupt, write a `1` to the appropriate bit in `INTR`.

12.8.4. Programmer's model

NOTE

The timer's tick (see Section 8.5) must be running for the timer to start counting. The SDK starts this tick as part of the platform initialisation code.

12.8.4.1. Reading the time

NOTE

Time here refers to the number of microseconds since the timer was started, not a clock. For a clock, see Section 12.10.

To read the 64-bit time, read `TIMELR` followed by `TIMEHR`. Reading `TIMELR` latches (stops) the value in `TIMEHR` until `TIMEHR` is read. Because RP2350 has 2 cores, it is unsafe to do this if the second core executes code that can also access the timer, or if the timer is read concurrently in an IRQ handler and in thread mode. If one core reads `TIMELR` followed by another core reading `TIMELR`, the value in `TIMEHR` isn't necessarily accurate. The example below shows the simplest form of getting the 64-bit time:

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/timer/timer_lowlevel/timer_lowlevel.c Lines 15 - 23

```
15 // Simplest form of getting 64 bit time from the timer.
16 // It isn't safe when called from 2 cores because of the latching
17 // so isn't implemented this way in the sdk
18 static uint64_t get_time(void) {
19     // Reading low latches the high value
20     uint32_t lo = timer_hw->timelr;
21     uint32_t hi = timer_hw->timehr;
22     return ((uint64_t) hi << 32u) | lo;
23 }
```

The SDK provides a `time_us_64` function that uses a more thorough method to get the 64-bit time, which makes use of the `TIMERAWH` and `TIMERAWL` registers. The `RAW` registers don't latch, making `time_us_64` safe to call from multiple cores at once.

SDK: https://github.com/raspberrypi/pico-sdk/blob/master/src/rp2_common/hardware_timer/timer.c Lines 57 - 73

```
57 uint64_t timer_time_us_64(timer_hw_t *timer) {
58     // Need to make sure that the upper 32 bits of the timer
59     // don't change, so read that first
60     uint32_t hi = timer->timerawh;
61     uint32_t lo;
62     do {
63         // Read the lower 32 bits
64         lo = timer->timerawl;
65         // Now read the upper 32 bits again and
66         // check that it hasn't incremented. If it has loop around
67         // and read the lower 32 bits again to get an accurate value
68         uint32_t next_hi = timer->timerawh;
69         if (hi == next_hi) break;
70         hi = next_hi;
71     } while (true);
72     return ((uint64_t) hi << 32u) | lo;
73 }
```

12.8.4.2. Set an alarm

The standalone timer example, `timer_lowlevel`, demonstrates how to set an alarm at a hardware level without the additional abstraction over the timer provided by SDK. To use these abstractions, see Section 12.8.4.4.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/timer/timer_lowlevel/timer_lowlevel.c Lines 27 - 74

```
27 // Use alarm 0
28 #define ALARM_NUM 0
29 #define ALARM_IRQ timer_hardware_alarm_get_irq_num(timer_hw, ALARM_NUM)
30
31 // Alarm interrupt handler
32 static volatile bool alarm_fired;
33
34 static void alarm_irq(void) {
35     // Clear the alarm irq
36     hw_clear_bits(&timer_hw->intr, 1u << ALARM_NUM);
37
38     // Assume alarm 0 has fired
39     printf("Alarm IRQ fired\n");
40     alarm_fired = true;
41 }
42
43 static void alarm_in_us(uint32_t delay_us) {
44     // Enable the interrupt for our alarm (the timer outputs 4 alarm irqs)
45     hw_set_bits(&timer_hw->inte, 1u << ALARM_NUM);
46     // Set irq handler for alarm irq
47     irq_set_exclusive_handler(ALARM_IRQ, alarm_irq);
48     // Enable the alarm irq
49     irq_set_enabled(ALARM_IRQ, true);
50     // Enable interrupt in block and at processor
51
52     // Alarm is only 32 bits so if trying to delay more
53     // than that need to be careful and keep track of the upper
54     // bits
55     uint64_t target = timer_hw->timerawl + delay_us;
56
57     // Write the lower 32 bits of the target time to the alarm which
58     // will arm it
59     timer_hw->alarm[ALARM_NUM] = (uint32_t) target;
```

```
60 }
61
62 int main() {
63     stdio_init_all();
64     printf("Timer lowlevel!\n");
65
66     // Set alarm every 2 seconds
67     while (1) {
68         alarm_fired = false;
69         alarm_in_us(1000000 * 2);
70         // Wait for alarm to fire
71         while (!alarm_fired);
72     }
73 }
```

12.8.4.3. Busy wait

If you don't want to use an alarm to wait for a period of time, use a while loop instead. The SDK provides various `busy_wait_` functions to do this:

SDK: https://github.com/raspberrypi/pico-sdk/blob/master/src/rp2_common/hardware_timer/timer.c Lines 77 - 122

```
77 void timer_busy_wait_us_32(timer_hw_t *timer, uint32_t delay_us) {
78     if (0 <= (int32_t)delay_us) {
79         // we only allow 31 bits, otherwise we could have a race in the loop below with
80         // values very close to 2^32
81         uint32_t start = timer->timerawl;
82         while (timer->timerawl - start < delay_us) {
83             tight_loop_contents();
84         }
85     } else {
86         busy_wait_us(delay_us);
87     }
88 }
89
90 void timer_busy_wait_us(timer_hw_t *timer, uint64_t delay_us) {
91     uint64_t base = timer_time_us_64(timer);
92     uint64_t target = base + delay_us;
93     if (target < base) {
94         target = (uint64_t)-1;
95     }
96     absolute_time_t t;
97     update_us_since_boot(&t, target);
98     timer_busy_wait_until(timer, t);
99 }
100
101 void timer_busy_wait_ms(timer_hw_t *timer, uint32_t delay_ms)
102 {
103     if (delay_ms <= 0x7fffffffu / 1000) {
104         timer_busy_wait_us_32(timer, delay_ms * 1000);
105     } else {
106         timer_busy_wait_us(timer, delay_ms * 1000ull);
107     }
108 }
109
110 void timer_busy_wait_until(timer_hw_t *timer, absolute_time_t t) {
111     uint64_t target = to_us_since_boot(t);
112     uint32_t hi_target = (uint32_t)(target >> 32u);
113     uint32_t hi = timer->timerawh;
114     while (hi < hi_target) {
```

```
115         hi = timer->timerawh;
116         tight_loop_contents();
117     }
118     while (hi == hi_target && timer->timerawl < (uint32_t) target) {
119         hi = timer->timerawh;
120         tight_loop_contents();
121     }
122 }
```

12.8.4.4. Complete example using SDK

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/timer/hello_timer/hello_timer.c Lines 11 - 57

```
11 volatile bool timer_fired = false;
12
13 int64_t alarm_callback(alarm_id_t id, __unused void *user_data) {
14     printf("Timer %d fired!\n", (int) id);
15     timer_fired = true;
16     // Can return a value here in us to fire in the future
17     return 0;
18 }
19
20 bool repeating_timer_callback(__unused struct repeating_timer *t) {
21     printf("Repeat at %lld\n", time_us_64());
22     return true;
23 }
24
25 int main() {
26     stdio_init_all();
27     printf("Hello Timer!\n");
28
29     // Call alarm_callback in 2 seconds
30     add_alarm_in_ms(2000, alarm_callback, NULL, false);
31
32     // Wait for alarm callback to set timer_fired
33     while (!timer_fired) {
34         tight_loop_contents();
35     }
36
37     // Create a repeating timer that calls repeating_timer_callback.
38     // If the delay is > 0 then this is the delay between the previous callback ending and the
   next starting.
39     // If the delay is negative (see below) then the next call to the callback will be exactly
   500ms after the
40     // start of the call to the last callback
41     struct repeating_timer timer;
42     add_repeating_timer_ms(500, repeating_timer_callback, NULL, &timer);
43     sleep_ms(3000);
44     bool cancelled = cancel_repeating_timer(&timer);
45     printf("cancelled... %d\n", cancelled);
46     sleep_ms(2000);
47
48     // Negative delay so means we will call repeating_timer_callback, and call it again
49     // 500ms later regardless of how long the callback took to execute
50     add_repeating_timer_ms(-500, repeating_timer_callback, NULL, &timer);
51     sleep_ms(3000);
52     cancelled = cancel_repeating_timer(&timer);
53     printf("cancelled... %d\n", cancelled);
54     sleep_ms(2000);
55     printf("Done\n");
```

```
    return 0;
}
```

12.8.5. List of registers

The `TIMER0` and `TIMER1` registers start at base addresses of `0x400b0000` and `0x400b8000` respectively (defined as TIMER0_BASE and TIMER1_BASE in SDK).

Table 1227. List of TIMER registers

Offset	Name	Info
0x00	TIMEHW	Write to bits 63:32 of time always write timelw before timehw
0x04	TIMELW	Write to bits 31:0 of time writes do not get copied to time until timehw is written
0x08	TIMEHR	Read from bits 63:32 of time always read timelr before timehr
0x0c	TIMELR	Read from bits 31:0 of time
0x10	ALARM0	Arm alarm 0, and configure the time it will fire. Once armed, the alarm fires when TIMER_ALARM0 == TIMELR. The alarm will disarm itself once it fires, and can be disarmed early using the ARMED status register.
0x14	ALARM1	Arm alarm 1, and configure the time it will fire. Once armed, the alarm fires when TIMER_ALARM1 == TIMELR. The alarm will disarm itself once it fires, and can be disarmed early using the ARMED status register.
0x18	ALARM2	Arm alarm 2, and configure the time it will fire. Once armed, the alarm fires when TIMER_ALARM2 == TIMELR. The alarm will disarm itself once it fires, and can be disarmed early using the ARMED status register.
0x1c	ALARM3	Arm alarm 3, and configure the time it will fire. Once armed, the alarm fires when TIMER_ALARM3 == TIMELR. The alarm will disarm itself once it fires, and can be disarmed early using the ARMED status register.
0x20	ARMED	Indicates the armed/disarmed status of each alarm. A write to the corresponding ALARMx register arms the alarm. Alarms automatically disarm upon firing, but writing ones here will disarm immediately without waiting to fire.
0x24	TIMERAWH	Raw read from bits 63:32 of time (no side effects)
0x28	TIMERAWL	Raw read from bits 31:0 of time (no side effects)
0x2c	DBGPAUSE	Set bits high to enable pause when the corresponding debug ports are active
0x30	PAUSE	Set high to pause the timer
0x34	LOCKED	Set locked bit to disable write access to timer Once set, cannot be cleared (without a reset)

Offset	Name	Info
0x38	SOURCE	Selects the source for the timer. Defaults to the normal tick configured in the ticks block (typically configured to 1 microsecond). Writing to 1 will ignore the tick and count clk_sys cycles instead.
0x3c	INTR	Raw Interrupts
0x40	INTE	Interrupt Enable
0x44	INTF	Interrupt Force
0x48	INTS	Interrupt status after masking & forcing

TIMER: TIMEHW Register

Offset: 0x00

Table 1228. TIMEHW Register

Bits	Description	Type	Reset
31:0	Write to bits 63:32 of time always write timelw before timehw	WF	0x00000000

TIMER: TIMELW Register

Offset: 0x04

Table 1229. TIMELW Register

Bits	Description	Type	Reset
31:0	Write to bits 31:0 of time writes do not get copied to time until timehw is written	WF	0x00000000

TIMER: TIMEHR Register

Offset: 0x08

Table 1230. TIMEHR Register

Bits	Description	Type	Reset
31:0	Read from bits 63:32 of time always read timelr before timehr	RO	0x00000000

TIMER: TIMELR Register

Offset: 0x0c

Table 1231. TIMELR Register

Bits	Description	Type	Reset
31:0	Read from bits 31:0 of time	RO	0x00000000

TIMER: ALARM0 Register

Offset: 0x10

Table 1232. ALARM0 Register

Bits	Description	Type	Reset
31:0	Arm alarm 0, and configure the time it will fire. Once armed, the alarm fires when TIMER_ALARM0 == TIMELR. The alarm will disarm itself once it fires, and can be disarmed early using the ARMED status register.	RW	0x00000000

TIMER: ALARM1 Register

Offset: 0x14

Table 1233. ALARM1 Register

Bits	Description	Type	Reset
31:0	Arm alarm 1, and configure the time it will fire. Once armed, the alarm fires when TIMER_ALARM1 == TIMELR. The alarm will disarm itself once it fires, and can be disarmed early using the ARMED status register.	RW	0x00000000

TIMER: ALARM2 Register

Offset: 0x18

Table 1234. ALARM2 Register

Bits	Description	Type	Reset
31:0	Arm alarm 2, and configure the time it will fire. Once armed, the alarm fires when TIMER_ALARM2 == TIMELR. The alarm will disarm itself once it fires, and can be disarmed early using the ARMED status register.	RW	0x00000000

TIMER: ALARM3 Register

Offset: 0x1c

Table 1235. ALARM3 Register

Bits	Description	Type	Reset
31:0	Arm alarm 3, and configure the time it will fire. Once armed, the alarm fires when TIMER_ALARM3 == TIMELR. The alarm will disarm itself once it fires, and can be disarmed early using the ARMED status register.	RW	0x00000000

TIMER: ARMED Register

Offset: 0x20

Table 1236. ARMED Register

Bits	Description	Type	Reset
31:4	Reserved.	-	-
3:0	Indicates the armed/disarmed status of each alarm. A write to the corresponding ALARMx register arms the alarm. Alarms automatically disarm upon firing, but writing ones here will disarm immediately without waiting to fire.	WC	0x0

TIMER: TIMERAWH Register

Offset: 0x24

Table 1237. TIMERAWH Register

Bits	Description	Type	Reset
31:0	Raw read from bits 63:32 of time (no side effects)	RO	0x00000000

TIMER: TIMERAWL Register

Offset: 0x28

Table 1238. TIMERAWL Register

Bits	Description	Type	Reset
31:0	Raw read from bits 31:0 of time (no side effects)	RO	0x00000000

TIMER: DBGPAUSE Register

Offset: 0x2c

Description

Set bits high to enable pause when the corresponding debug ports are active

Table 1239. DBGPAUSE Register

Bits	Description	Type	Reset
31:3	Reserved.	-	-
2	**DBG1**: Pause when processor 1 is in debug mode	RW	0x1
1	**DBG0**: Pause when processor 0 is in debug mode	RW	0x1
0	Reserved.	-	-

TIMER: PAUSE Register

Offset: 0x30

Table 1240. PAUSE Register

Bits	Description	Type	Reset
31:1	Reserved.	-	-
0	Set high to pause the timer	RW	0x0

TIMER: LOCKED Register

Offset: 0x34

Table 1241. LOCKED Register

Bits	Description	Type	Reset
31:1	Reserved.	-	-
0	Set locked bit to disable write access to timer Once set, cannot be cleared (without a reset)	RW	0x0

TIMER: SOURCE Register

Offset: 0x38

Description

Selects the source for the timer. Defaults to the normal tick configured in the ticks block (typically configured to 1 microsecond). Writing to 1 will ignore the tick and count clk_sys cycles instead.

Table 1242. SOURCE Register

Bits	Description	Type	Reset
31:1	Reserved.	-	-
0	**CLK_SYS**	RW	0x0

Bits	Description	Type	Reset
	Enumerated values:		
	0x0 → TICK		
	0x1 → CLK_SYS		

TIMER: INTR Register

Offset: 0x3c

Description

Raw Interrupts

Table 1243. INTR Register

Bits	Description	Type	Reset
31:4	Reserved.	-	-
3	**ALARM_3**	WC	0x0
2	**ALARM_2**	WC	0x0
1	**ALARM_1**	WC	0x0
0	**ALARM_0**	WC	0x0

TIMER: INTE Register

Offset: 0x40

Description

Interrupt Enable

Table 1244. INTE Register

Bits	Description	Type	Reset
31:4	Reserved.	-	-
3	**ALARM_3**	RW	0x0
2	**ALARM_2**	RW	0x0
1	**ALARM_1**	RW	0x0
0	**ALARM_0**	RW	0x0

TIMER: INTF Register

Offset: 0x44

Description

Interrupt Force

Table 1245. INTF Register

Bits	Description	Type	Reset
31:4	Reserved.	-	-
3	**ALARM_3**	RW	0x0
2	**ALARM_2**	RW	0x0
1	**ALARM_1**	RW	0x0
0	**ALARM_0**	RW	0x0

TIMER: INTS Register

Offset: 0x48

Description

Interrupt status after masking & forcing

Table 1246. INTS Register

Bits	Description	Type	Reset
31:4	Reserved.	-	-
3	**ALARM_3**	RO	0x0
2	**ALARM_2**	RO	0x0
1	**ALARM_1**	RO	0x0
0	**ALARM_0**	RO	0x0

12.9. Watchdog

12.9.1. Overview

The watchdog is a countdown timer which can be configured to reset selected components when it reaches zero. In normal operation it is periodically loaded with a non-zero value to prevent the reset occuring. If the chip locks up or software gets stuck in a loop, the reset allows recovery.

The watchdog is reset by any chip-level reset (see Section 7.3). The sources of the chip-level reset are:

- Power-On Reset (POR)
- Brown-out Detection (BOD)
- External Reset (from the RUN pin)
- Debugger Reset Request
- Rescue Debug Port Request
- Watchdog - a chip-level reset triggered by the Watchdog will reset the Watchdog
- SWCORE powerdown
- Glitch Detector
- Debugger HZD Reset Request

These are described in Section 7.3.3.

12.9.2. Changes from RP2040

On RP2040, the watchdog contained a tick generator used to generate a 1µs tick for the watchdog. This was also distributed to the system timer. On RP2350, the watchdog instead takes a tick input from the system-level ticks block. See Section 8.5.

As on RP2040 the watchdog can trigger a PSM (Power-on State Machine) sequence to reset system components or it can be used to reset selected subsystem components. On RP2350, the watchdog can also trigger a chip level reset.

12.9.3. Watchdog counter

The watchdog counter is loaded by the LOAD register. The current value can be seen in CTRL.TIME.

12.9.4. Control watchdog reset levels

To control the level of reset triggered by a watchdog event, use the registers outside the watchdog register block:

- `POWMAN_WATCHDOG` allows the watchdog to trigger chip level resets
- `PSM_WDSEL` allows the watchdog to trigger system resets by running a full or partial PSM sequence (Power-on State Machine)
- `RESETS_WDSEL` allows the watchdog to trigger subsystem resets

These are described in the Resets section, see Chapter 7.

12.9.5. Scratch registers

The watchdog contains eight 32-bit scratch registers that can store information between soft resets of the chip. The scratch registers reset when:

- the watchdog is used to to trigger a chip level reset
- a `rst_n_run` event occurs, triggered by toggling the RUN pin or cycling the digital core supply (DVDD)

The bootrom checks the watchdog scratch registers for a magic number on boot. You can use this to soft reset the chip into user-specified code. See Section 5.2.4 for more information.

NOTE

Additional general-purpose scratch registers are available in POWMAN SCRATCH0 through SCRATCH7. These registers also survive power cycling the switched core domain.

12.9.6. Programmer's model

The SDK provides a `hardware_watchdog` driver to control the watchdog.

12.9.6.1. Enabling the watchdog

SDK: https://github.com/raspberrypi/pico-sdk/blob/master/src/rp2_common/hardware_watchdog/watchdog.c Lines 47 - 76

```
47 // Helper function used by both watchdog_enable and watchdog_reboot
48 void _watchdog_enable(uint32_t delay_ms, bool pause_on_debug) {
49     valid_params_if(HARDWARE_WATCHDOG, delay_ms <= WATCHDOG_LOAD_BITS / (1000 *
   WATCHDOG_XFACTOR));
50     hw_clear_bits(&watchdog_hw->ctrl, WATCHDOG_CTRL_ENABLE_BITS);
51
52     // Reset everything apart from ROSC and XOSC
53     hw_set_bits(&psm_hw->wdsel, PSM_WDSEL_BITS & ~(PSM_WDSEL_ROSC_BITS |
   PSM_WDSEL_XOSC_BITS));
54
55     uint32_t dbg_bits = WATCHDOG_CTRL_PAUSE_DBG0_BITS |
56                         WATCHDOG_CTRL_PAUSE_DBG1_BITS |
57                         WATCHDOG_CTRL_PAUSE_JTAG_BITS;
58
59     if (pause_on_debug) {
60         hw_set_bits(&watchdog_hw->ctrl, dbg_bits);
61     } else {
62         hw_clear_bits(&watchdog_hw->ctrl, dbg_bits);
63     }
64
```

```
    if (!delay_ms) {
        hw_set_bits(&watchdog_hw->ctrl, WATCHDOG_CTRL_TRIGGER_BITS);
    } else {
        load_value = delay_ms * (1000 * WATCHDOG_XFACTOR);
        if (load_value > WATCHDOG_LOAD_BITS)
            load_value = WATCHDOG_LOAD_BITS;

        watchdog_update();

        hw_set_bits(&watchdog_hw->ctrl, WATCHDOG_CTRL_ENABLE_BITS);
    }
}
```

12.9.6.2. Updating the watchdog counter

SDK: https://github.com/raspberrypi/pico-sdk/blob/master/src/rp2_common/hardware_watchdog/watchdog.c Lines 24 - 28

```
24 static uint32_t load_value;
25
26 void watchdog_update(void) {
27     watchdog_hw->load = load_value;
28 }
```

12.9.6.3. Usage

The Pico Examples repository provides a `hello_watchdog` example that uses the `hardware_watchdog` to demonstrate use of the watchdog.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/watchdog/hello_watchdog/hello_watchdog.c Lines 11 - 33

```
11 int main() {
12     stdio_init_all();
13
14     if (watchdog_enable_caused_reboot()) {
15         printf("Rebooted by Watchdog!\n");
16         return 0;
17     } else {
18         printf("Clean boot\n");
19     }
20
21     // Enable the watchdog, requiring the watchdog to be updated every 100ms or the chip will
   reboot
22     // second arg is pause on debug which means the watchdog will pause when stepping through
   code
23     watchdog_enable(100, 1);
24
25     for (uint i = 0; i < 5; i++) {
26         printf("Updating watchdog %d\n", i);
27         watchdog_update();
28     }
29
30     // Wait in an infinite loop and don't update the watchdog so it reboots us
31     printf("Waiting to be rebooted by watchdog\n");
32     while(1);
33 }
```

12.9.7. List of registers

The watchdog registers start at a base address of `0x400d8000` (defined as WATCHDOG_BASE in SDK).

Table 1247. List of WATCHDOG registers

Offset	Name	Info
0x00	CTRL	Watchdog control The rst_wdsel register determines which subsystems are reset when the watchdog is triggered. The watchdog can be triggered in software.
0x04	LOAD	Load the watchdog timer. The maximum setting is 0xffffff which corresponds to approximately 16 seconds.
0x08	REASON	Logs the reason for the last reset. Both bits are zero for the case of a hardware reset. Additionally, as of RP2350, a debugger warm reset of either core (SYSRESETREQ or hartreset) will also clear the watchdog reason register, so that software loaded under the debugger following a watchdog timeout will not continue to see the timeout condition.
0x0c	SCRATCH0	Scratch register. Information persists through soft reset of the chip.
0x10	SCRATCH1	Scratch register. Information persists through soft reset of the chip.
0x14	SCRATCH2	Scratch register. Information persists through soft reset of the chip.
0x18	SCRATCH3	Scratch register. Information persists through soft reset of the chip.
0x1c	SCRATCH4	Scratch register. Information persists through soft reset of the chip.
0x20	SCRATCH5	Scratch register. Information persists through soft reset of the chip.
0x24	SCRATCH6	Scratch register. Information persists through soft reset of the chip.
0x28	SCRATCH7	Scratch register. Information persists through soft reset of the chip.

WATCHDOG: CTRL Register

Offset: 0x00

Description

Watchdog control
The rst_wdsel register determines which subsystems are reset when the watchdog is triggered.
The watchdog can be triggered in software.

Table 1248. CTRL Register

Bits	Description	Type	Reset
31	**TRIGGER**: Trigger a watchdog reset	SC	0x0
30	**ENABLE**: When not enabled the watchdog timer is paused	RW	0x0
29:27	Reserved.	-	-
26	**PAUSE_DBG1**: Pause the watchdog timer when processor 1 is in debug mode	RW	0x1

Bits	Description	Type	Reset
25	**PAUSE_DBG0**: Pause the watchdog timer when processor 0 is in debug mode	RW	0x1
24	**PAUSE_JTAG**: Pause the watchdog timer when JTAG is accessing the bus fabric	RW	0x1
23:0	**TIME**: Indicates the time in usec before a watchdog reset will be triggered	RO	0x000000

WATCHDOG: LOAD Register

Offset: 0x04

Table 1249. LOAD Register

Bits	Description	Type	Reset
31:24	Reserved.	-	-
23:0	Load the watchdog timer. The maximum setting is 0xffffff which corresponds to approximately 16 seconds.	WF	0x000000

WATCHDOG: REASON Register

Offset: 0x08

Description

Logs the reason for the last reset. Both bits are zero for the case of a hardware reset.

Additionally, as of RP2350, a debugger warm reset of either core (SYSRESETREQ or hartreset) will also clear the watchdog reason register, so that software loaded under the debugger following a watchdog timeout will not continue to see the timeout condition.

Table 1250. REASON Register

Bits	Description	Type	Reset
31:2	Reserved.	-	-
1	**FORCE**	RO	0x0
0	**TIMER**	RO	0x0

WATCHDOG: SCRATCH0, SCRATCH1, …, SCRATCH6, SCRATCH7 Registers

Offsets: 0x0c, 0x10, …, 0x24, 0x28

Table 1251. SCRATCH0, SCRATCH1, …, SCRATCH6, SCRATCH7 Registers

Bits	Description	Type	Reset
31:0	Scratch register. Information persists through soft reset of the chip.	RW	0x00000000

12.10. Always-on timer

12.10.1. Overview

The always-on timer (AON Timer) is the only timer that operates in all power modes. It can be used as a real-time counter or an interval timer and incorporates an alarm which can be used to trigger a power-up event or an interrupt. It incorporates a 64-bit counter intended to count 1ms ticks, but the tick generator can be configured to run faster or slower if required. Note that the AON Timer tick generator is independent of all other tick generators on the chip.

The default tick source is the 32kHz on-chip low-power oscillator (LPOSC), see Section 8.4. The LPOSC frequency is not precise and may vary with voltage and temperature. When the chip core is powered, the tick source can be switched to the on-chip crystal oscillator (XOSC) for greater precision. If greater precision is also required when the chip core is

unpowered, then a 32kHz clock or a 1ms tick can be supplied from an external source. Alternatively, the AON Timer can be synchronised to an external 1Hz source.

The AON Timer is integrated with the power manager (POWMAN) and shares the POWMAN register block. Writes are limited to 16 bits because a key (`0x5afe`) is required in the top 16 bits to prevent erroneous writes from locking up the chip. Most AON Timer registers can be enabled for write by Non-secure software, unlike other POWMAN registers. However, the registers used to select an external clock, select an external tick source, and enable power-up on alarm can only be written by Secure software.

12.10.2. Changes from RP2040

The RP2040 Real Time Clock (RTC) is not used in RP2350. Instead, RP2350 has a timer in the Always-On power domain which is used for scheduling power-up events and can also be used as a real-time counter. The AON Timer works differently from the RP2040 RTC. It counts milliseconds to 64 bits and this value can be used to calculate the date and time in software if required.

12.10.3. Accessing the AON Timer

To start and stop the AON Timer, write to TIMER.RUN.

To read the current 64-bit AON Timer value, use the following 2 × 32-bit read-only registers:

- READ_TIME_UPPER
- READ_TIME_LOWER

Because the AON Timer can increment during a read, use the following procedure to protect against erroneous reads:

1. Read READ_TIME_UPPER
2. Read READ_TIME_LOWER
3. Read READ_TIME_UPPER
4. If the READ_TIME_UPPER value changes between steps 1 and 3, repeat the whole procedure

When used as a real time clock, the 64-bit time value is set using 4 × 16-bit registers. These registers can only be written when the AON Timer is stopped by writing a 0 to TIMER.RUN:

- SET_TIME_63TO48
- SET_TIME_47TO32
- SET_TIME_31TO16
- SET_TIME_15TO0

These registers cannot be used to read the time value.

When used as an interval timer, write a 1 to TIMER.CLEAR to clear the timer value. It is not necessary to stop the AON Timer to do this. The TIMER.CLEAR register is self-clearing: it returns to 0 when the operation completes. This allows easy implementation of an alarm that wakes the chip or generates an interrupt at regular intervals.

12.10.4. Using the alarm

To set the alarm time, use the following 4 × 16-bit registers:

- ALARM_TIME_63TO48
- ALARM_TIME_47TO32

- ALARM_TIME_31TO16
- ALARM_TIME_15TO0

To avoid false alarms, disable the alarm before setting the alarm time.

To enable the alarm, use TIMER.ALARM_ENAB.

When the alarm fires, the AON Timer sets the alarm status flag TIMER.ALARM.

To clear the alarm status flag, write a 1 to the alarm status flag.

To configure the alarm to trigger a power-up, set TIMER.PWRUP_ON_ALARM. This feature is not available to Non-secure code.

The alarm can be configured to trigger an interrupt. The interrupt is handled in the standard way using the following register fields:

- INTR.TIMER - raw interrupt
- INTE.TIMER - interrupt enable
- INTF.TIMER - force interrupt
- INTS.TIMER - interrupt status

12.10.5. Selecting the AON Timer tick source

The AON Timer indicates the current configuration with read-only flags. Table 1252 provides a list of sources supported by the 1kHz AON Timer tick.

Table 1252. AON Timer tick generators

Tick source	Read-only flag
LPOSC clock division	TIMER.USING_LPOSC
XOSC clock division	TIMER.USING_XOSC
external 1kHz tick	TIMER.USING_GPIO_1KHZ

NOTE

The LPOSC clock can be substituted by an external 32kHz clock.

12.10.5.1. Using LPOSC as the AON Timer tick source

LPOSC is the default source and can be used in all power modes. It nominally runs at 32.768kHz and can only be tuned to 1% accuracy. The AON Timer derives the 1ms tick from the LPOSC using a 6.16 bit fractional divider whose divisor is initialised to 32.768. The divisor can be modified to achieve greater accuracy. Because the LPOSC frequency varies with supply voltage and temperature, accuracy is limited unless supply voltage and temperature are stable. To modify the divisor, write to the following registers:

- LPOSC_FREQ_KHZ_INT (default value: 32)
- LPOSC_FREQ_KHZ_FRAC (default value: 0.768)

These registers should only be written when TIMER.RUN = 0 or TIMER.USING_LPOSC = 0.

If the tick source is not LPOSC, you can switch it back to LPOSC by writing a 1 to TIMER.USE_LPOSC. It is not necessary to stop the AON Timer to do this. The newly selected tick will be synchronised to the current tick, so the operation may take up to 1 tick cycle (1ms in normal operation). When the operation is complete, TIMER.USE_LPOSC will self-clear and TIMER.USING_LPOSC will be set. Due to sampling, a small error of up to 2 periods of the newly selected clock will be subtracted from the time. When switching to LPOSC at 32kHz, an error of up to 62µs will be subtracted.

12.10.5.2. Using an external clock in place of LPOSC

If LPOSC isn't sufficiently accurate, an external 32.768kHz clock can be used. This will be multiplexed onto the internal low-power clock and will therefore drive all components that are driven by that clock, including the power sequencer components. The external clock can be used in all power modes. When an external clock is in use, you can stop the LPOSC (see Section 8.4).

To select an external 32kHz clock:

1. Configure the GPIO source as described in Section 12.10.7.
2. Switch to the external LPOSC by setting EXT_TIME_REF.DRIVE_LPCK. This register should only be written when TIMER.RUN = 0 and the power sequencer is inactive. You can only write to this register from Secure code.

The external 32kHz clock replaces the clock from LPOSC. Therefore the same registers are used for AON Timer configuration (see Section 12.10.5.1):

- TIMER.USE_LPOSC
- TIMER.USING_LPOSC
- LPOSC_FREQ_KHZ_INT
- LPOSC_FREQ_KHZ_FRAC

12.10.5.3. Using the XOSC as the AON Timer tick source

The XOSC clock is provided via the reference clock (`clk_ref`). The user must ensure the reference clock is being driven from the XOSC before selecting it as the source of the AON Timer tick. This is the normal configuration following boot. To check, look for CLK_REF_SELECTED = `0x4`. The reference clock may be a divided version of the XOSC. The divisor defaults to 1 and can be read from CLK_REF_DIV.INT. If the chip is operated with a faster XOSC, the clock sent to the AON Timer must not exceed 29MHz.

The AON Timer derives the 1ms tick from the XOSC using a 16.16 bit fractional divider whose divisor is initialised to 12000.0. This assumes a 12MHz crystal is used and the reference clock divisor is 1. If that is not the case, the divisor in the AON Timer can be modified by writing to the following registers:

- XOSC_FREQ_KHZ_INT (default value: 12000)
- XOSC_FREQ_KHZ_FRAC (default value: 0)

These registers should only be written when TIMER.RUN = 0 or TIMER.USING_XOSC = 0.

To select the XOSC as the AON Timer tick source, write a 1 to TIMER.USE_XOSC. It is not necessary to stop the AON Timer to do this. The newly selected tick will be synchronised to the current tick, so the operation may take up to 1 tick cycle (1ms in normal operation). When the operation is complete TIMER.USE_XOSC will self-clear and TIMER.USING_XOSC will be set. Due to sampling, a small error of up to 2 periods of the newly selected clock will be subtracted from the time. When switching to XOSC at 12MHz an error of up to 167ns will be subtracted.

When the chip core is powered down the XOSC will stop. If TIMER.USING_XOSC is set, the power-down sequencer automatically reverts to TIMER.USING_LPOSC before the XOSC stops.

12.10.5.4. Using an external 1ms tick source

To select an external 1ms tick source, configure the GPIO source as described in Section 12.10.7. Then, write a 1 to TIMER.USE_GPIO_1KHZ. It is not necessary to stop the AON Timer to do this, however the newly selected tick will not be synchronised to the current tick, so the operation so the operation will advance the time by up to 1ms. If using an external 1ms tick it is recommended to set the time after selecting the source. When the operation is complete TIMER.USE_GPIO_1KHZ will self-clear and TIMER.USING_GPIO_1KHZ will be set.

The tick is triggered from the falling edge of the selected GPIO. For correct sampling, the GPIO pulse width and interval must both be greater than the period of LPOSC (>31us). This limits the maximum frequency of the external tick to

16kHz.

The external 1ms tick can be used in all power modes.

12.10.6. Synchronising the AON timer to an external 1Hz clock

In applications that use GPS, a 1s tick may be available. This can be used to synchronise the AON Timer and thus compensate for inaccuracy in the LPOSC frequency. It can be used with any tick source, but there is little to be gained if the selected source is already reasonably accurate.

If the LPOSC is fast, the ms counter pauses at a 1 second step until the 1s tick is received. If the LPOSC is slow, the 1s tick causes the ms counter to run very quickly until reaching the 1 second step. This ensures that all ms values are counted, ensuring that any alarm set to ms precision will fire. A more sophisticated synchronisation method can be implemented in software.

To use the hardware synchronisation feature, configure the GPIO source as described in Section 12.10.7. Then, enable the feature by writing a 1 to TIMER.USE_GPIO_1HZ. This can be set at any time, it is not necessary to stop the AON Timer. When the operation is complete TIMER.USE_GPIO_1HZ will self-clear and TIMER.USING_GPIO_1HZ will be set.

The tick is triggered from the falling edge of the selected GPIO. For correct sampling, the GPIO pulse width and interval must be greater than the period of LPOSC (>31us).

The external 1s tick can be used in all power modes.

12.10.7. Using an external clock or tick from GPIO

The following features use a GPIO as a clock or a tick:

- external 32kHz clock source
- external 1kHz tick
- external 1Hz tick

Only 4 GPIOs are available for these features. You can only select one, because they share the same GPIO selection logic. The set of 4 GPIOs differs between package types. The selection is controlled by a 2-bit register field.

The AON Timer uses the following GPIOs:

- EXT_TIME_REF.SOURCE_SEL = 0 → GPIO12
- EXT_TIME_REF.SOURCE_SEL = 1 → GPIO20
- EXT_TIME_REF.SOURCE_SEL = 2 → GPIO14
- EXT_TIME_REF.SOURCE_SEL = 3 → GPIO22

12.10.8. Using a tick faster than 1ms

The tick rate can be increased by scaling the value written to the LPOSC and XOSC frequency registers. For example, if the frequency value is divided by 4 then the AON Timer will tick 4 times per ms. The minimum value that can be written to the frequency registers is 2.0, therefore the maximum upscaling using this method with LPOSC is 16, giving a time resolution of 1/16th of 1 ms (= 62.5us).

As described previously, the external tick is limited to 16kHz, so the maximum upscaling using this method is also 16. This gives a time resolution of 1/16th of 1 ms (62.5µs).

These limitations can be overcome either by using a faster external clock (see Section 12.10.5.2) or keeping the chip core powered so the AON Timer is always running from the XOSC. If a faster external clock is used then the power sequencer timings will also need to be adjusted.